CENTRO UNIVERSITÁRIO FEI

Avenida Humberto de Alencar Castelo Branco, 3972, CEP: 09850-901 São Bernardo do Campo
Telefone: (011) 4353-2900 Fax (011) 4109-5994

Curso de Ciências da Computação

Registro e validação de resultado de partida no FCHub

Análise de Serviços

Autores:	Data de emissão:
Laura de Souza Parente RA: 22.123.033-7 Rafael Takahagi Mendes RA: 22.126.084-7 João Pedro Lopes Santana Villas Boas RA: 22.125.065-7 Gabriel Koiama RA: 22.125.067-3	06/04/2026
Revisor:	Data de revisão

Índice

1 OBJETIVO DO DOCUMENTO 3

2 DEFINIÇÃO DE SERVIÇOS CANDIDATOS 3

2.1 Identificação de Operações Candidatas 3

2.2 Serviços Candidatos Identificados 3

2.3 Descrição de Serviços 3

1 Objetivo do Documento

Este documento tem como objetivo identificar e descrever os **serviços candidatos** da plataforma de gerenciamento de torneios esportivos.

A análise busca identificar operações do sistema a partir do processo de negócio e agrupá-las em serviços reutilizáveis, que poderão compor a arquitetura da aplicação.

2 Definição de Serviços Candidatos

2.1 Identificação de Operações Candidatas

Tarefa do Processo To Be	Operação Candidata
Validar credenciais do usuário	validarCredenciais()
Exibir mensagem de erro	exibirMensagem()
Validar partida e resultado informado	validarResultadoPartida()
Solicitar correção do resultado	solicitarCorrecaoResultado()
Registrar resultado como aguardando confirmação	registrarResultadoPendente()
Aguardar resposta do adversário	controlarPrazoResposta()
Atribuir vitória ao jogador que registrou o resultado	atribuirVitoriaJogador()
Oficializar resultado por ausência de resposta	oficializarResultadoAutomatico()
Atualizar chaveamento	atualizarChaveamento()
Atualizar ranking	atualizarRanking()
Sincronizar chaveamento no challonge	sincronizarChaveamentoChallonge()
Enviar notificação de confirmação aos jogadores	enviarNotificacaoConfirmacaoJogadores()

Serviços Candidatos Identificados

Agrupar as operações candidatas por entidade e/ou contexto lógico e identificar os serviços candidatos.
Usar a notação de classes da UML para descrever cada serviço com as suas operações. (Vide exemplo)

<<Interface>> Autenticacao Service
validarCredenciais()

<<Interface>> Notificacao Service
exibirMensagem()

<<Interface>> ValidacaoResultado Service
validarResultadoPartida()

<<Interface>> SolicitaçãoCorreção Service
solicitarCorrecaoResultado()

<<Interface>> AtualizarChaveamentoService
atualizarChaveamento()

<<Interface>> RegistrarResultado Service
registrarResultadoPendente()

<<Interface>> OficializarResultadoPorAusencia Service
oficializarResultadoAutomatico ()

<<Interface>> ControlePrazo Service
controlarPrazoResposta()

<<Interface>> AtribuirVitória Service
atribuirVitoriaJogador()

<<Interface>> Ranking Service
+atualizarRanking()

<<Interface>> IntegracaoChallongeService
+sincronizarChaveamentoChallon ge()

<<Interface>> NotificacaoService
+enviarNotificacaoConfirmacaoJogadores()

Descrição de Serviços

Descreva cada serviço, indicando: objetivo, escolha uma das operações, para a operação detalhe dados de entrada, dados de saída e o passo a passo do fluxo lógico.

Serviço Autenticacao Service
Objetivo: Validar as credenciais informadas pelo usuário
Operação: validarCredenciais()
Dados de entrada: login do usuário senha do usuário
Dados de saída: status da validação das credenciais
Descrição da lógica: 1. O sistema recebe o login informado. 2. O sistema recebe a senha informada. 3. O sistema compara as credenciais com os dados cadastrados. 4. O sistema retorna o status da validação.

Serviço Notificacao Service
Objetivo: Apresentar ao usuário uma mensagem de retorno quando ocorrer erro no processo
Operação: exibirMensagem()
Dados de entrada: tipo da mensagem conteúdo da mensagem
Dados de saída: mensagem exibida ao usuário
Descrição da lógica: 1. O sistema identifica a necessidade de apresentar uma mensagem de erro. 2. O sistema recupera o texto da mensagem correspondente. 3. O sistema exibe a mensagem ao usuário.

Serviço ValidacaoResultado Service
Objetivo: Verificar se o resultado informado para a partida é válido de acordo com as regras do sistema.
Operação: `validarResultadoPartida()`
Dados de entrada: identificação da partida placar informado
Dados de saída: status da validação do resultado
Descrição da lógica: 1. O sistema recebe a identificação da partida. 2. O sistema recebe o resultado informado. 3. O sistema verifica se a partida e o resultado podem ser validados. 4. O sistema retorna o status da validação.

Serviço PrazoRespostaService
Objetivo:Controlar o prazo de resposta do adversário.
Operação: controlarPrazoResposta()
Dados de entrada: identificação da partida **data e hora de início do prazo** **tempo limite para resposta**
Dados de saída: status do prazo de resposta
Descrição da lógica: O sistema recebe a identificação da partida. O sistema registra o início do prazo de resposta. O sistema acompanha o tempo disponível para manifestação do adversário. O sistema verifica se o prazo ainda está válido ou se expirou. O sistema retorna o status do prazo de resposta.

Serviço ResultadoAutomaticoService
Objetivo: Atribuir vitória ao jogador que registrou o resultado.
Operação: atribuirVitoriaJogador()
Dados de entrada: **identificação da partida** **identificação do jogador que registrou o resultado**
Dados de saída: vitória atribuída ao jogador status da atribuição do resultado
Descrição da lógica: O sistema recebe a identificação da partida. O sistema identifica o jogador que registrou o resultado. O sistema verifica a ausência de resposta do adversário dentro do prazo. O sistema atribui a vitória ao jogador correspondente. O sistema retorna o status da atribuição do resultado.

Serviço Oficializar resultado por ausência de resposta
Objetivo: Oficializar o resultado de uma partida caso o tempo limite para confirmar o resultado seja excedido
Operação: oficializarResultadoAutomatico()
Dados de entrada: Placar da Partida
Dados de saída: Partida Finalizada
Descrição da lógica: 1. Jogador A registra o resultado da partida 2. Aguarda a resposta do jogador B 3. Jogador B não responde no período de 30 minutos 4. Vitória é concedida ao jogador que registrou o resultado 5. Resultado é oficializado

Serviço Atualizar chaveamento
Objetivo: Atualizar o chaveamento do torneio
Operação: atualizarChaveamento()
Dados de entrada: Resultado Oficializado
Dados de saída: Chaveamento atualizado
Descrição da lógica: 1. Resultado oficializado 2. Jogador vencedor avança de fase 3. Jogador derrotado é eliminado 4. Chaveamento atualizado

Serviço: Registrar resultado como aguardando confirmação
Objetivo: Realizar o registro
Operação: registrarResultadoPendente()
Dados de entrada: Partida, Placar Informado
Dados de saída: Status "Aguardando Confirmação"
Descrição da lógica: 4. Jogador A registra resultado 5. Partida definida com status de “Aguardando confirmação”

Serviço: Solicitar correção do resultado
Objetivo: Usuário realizar uma solicitação ao sistema para corrigir o resultado informado pelo outro jogador
Operação: solicitarCorrecaoResultado()
Dados de entrada: Recusa do resultado
Dados de saída: Confirmação de solicitação enviada
Descrição da lógica: 6. Identifica que o resultado informado pelo Jogador A não condiz com a validação do sistema ou do Jogador B. 7. Abre um chamado de correção para a partida. 8. Notifica o Jogador A para reinserir ou ajustar os dados da partida.

Serviço: RankingService
Objetivo: Atualizar o ranking dos jogadores
Operação: atualizarRanking()
Dados de entrada: ● identificação dos jogadores ● resultado oficial da partida
Dados de saída: ● ranking atualizado
Descrição da lógica: 1. O sistema recebe o resultado oficial da partida. 2. O sistema identifica os jogadores envolvidos. 3. O sistema atualiza a pontuação ou posição dos jogadores no ranking. 4. O sistema registra o ranking atualizado.

Serviço: IntegracaoChallongeService
Objetivo: Sincronizar o chaveamento do torneio no Challonge.
Operação: sincronizarChaveamentoChallonge()
Dados de entrada: ● identificação do torneio ● dados atualizados do chaveamento
Dados de saída: ● status da sincronização do chaveamento
Descrição da lógica: 1. O sistema recebe a identificação do torneio. 2. O sistema recupera os dados atualizados do chaveamento. 3. O sistema envia as informações para o Challonge. 4. O sistema retorna o status da sincronização.

Serviço: NotificacaoService
Objetivo: Enviar notificação de confirmação aos jogadores.
Operação: enviarNotificacaoConfirmacaoJogadores()
Dados de entrada: ● identificação dos jogadores ● dados da solicitação de confirmação
Dados de saída: ● status do envio da notificação
Descrição da lógica: 1. O sistema identifica os jogadores que devem receber a notificação. 2. O sistema monta a mensagem de confirmação. 3. O sistema envia a notificação aos jogadores. 4. O sistema retorna o status do envio.